Tarea 1

Alineamiento de lecturas

Pablo Salazar-Méndez

2026-03-01

Introducción

El alineamiento de secuencias permite la inferencia de transcritos expresados al mapear las lecturas obtenidas a partir de un experimento de RNA-seq a un genoma de referencia. En esta tarea se realizará el alineamiento de dichas lecturas utilizando los alineadores STAR y HISAT2, con el fin de comparar su rendimiento y calidad de alineamiento.

Para ello, se descargaron datos de RNA-seq de la serie GSE132040, extrayendo los SRR correspondientes a muestras de riñón de ratones machos de 3 y 18 meses de edad. Luego, se realizó la evaluación de calidad y limpieza de las lecturas utilizando FastQC y Trimmomatic, respectivamente, para posteriormente realizar el alineamiento utilizando ambos alineadores. Finalmente, se compararon los resultados obtenidos en términos de porcentaje de lecturas alineadas y tiempo de ejecución.

Resultados

Descarga de datos

La descarga de los datos se realizó utilizando las herramientas `wget` para obtener los metadatos y `prefetch` junto con `fasterq-dump` para obtener los archivos `fastq` a partir de los archivos SRA. Se descargaron un **total de 7 archivos SRR** correspondientes a muestras de **riñón de ratones machos de 3 y 18 meses** de edad.

```python
#!/usr/bin/env python3

from datetime import datetime
import subprocess as sb
import pandas as pd
import os
import re

# Ubicando en el root del proyecto
if (pwd := os.getcwd()).split("/")[-1] == "src":
    os.chdir("..")
    print(f"Working directory changed to: {pwd}", flush=True)
else:
    print(f"Current working directory: {pwd}", flush=True)

# Descarga de datos SRA
file = "./data/GSE132040_MACA_Bulk_metadata.csv"
url =
 ↪  "https://ftp.ncbi.nlm.nih.gov/geo/series/GSE132nnn/GSE132040/suppl/GSE132040_MACA_
if not os.path.exists(file):
    print("Downloading metadata from GEO...", flush=True)
    sb.run(
        # Parámetros de ejecución de wget
        ["wget", url, "-O", file],
        check=True
    )

# Carga de metadatos
metadata = pd.read_csv(file)
del file, url

# Filtrado
patt_tissue = re.compile(r"^kidney(.?)+", re.IGNORECASE)
metadata = metadata[
    ((metadata["characteristics: age"] == "3") |
 ↪  (metadata["characteristics: age"] == "18")) &
    ((metadata["characteristics: sex"] == "m") & (metadata["source
 ↪  name"].astype(str).str.match(patt_tissue)))
]
SRR_files = list(metadata["raw file"]) # Extracción de los nombres de
 ↪  los archivos SRR
```

```python
del pwd, patt_tissue, metadata

print(f"SRR files to download: {SRR_files}", flush=True)
print("-" * 50, flush=True)

# Generación de directorios
sra_dir = "./data/sra/"
fastq_dir = "./data/fastq/"

os.makedirs(sra_dir, exist_ok=True)
os.makedirs(fastq_dir, exist_ok=True)
os.makedirs("./tmp/", exist_ok=True)

# Obtención de archivos SRA y conversión a FASTQ
for SRR in SRR_files:

    # Secuencias PE
    sra_file = os.path.join(sra_dir, SRR, f"{SRR}.sra")
    fastq_1 = os.path.join(fastq_dir, f"{SRR}_1.fastq")
    fastq_2 = os.path.join(fastq_dir, f"{SRR}_2.fastq")
    fastq_single = os.path.join(fastq_dir, f"{SRR}.fastq")

    # De no existir el archivo SRA, se descarga utilizando prefetch
    if not os.path.exists(sra_file):
        print(f"[{datetime.now().strftime('%d-%H:%M:%S')}] Downloading
        ↪ {SRR}...", flush=True)
        sb.run(["prefetch", SRR, "--output-directory", sra_dir],
↪   check=True)

    print(f"[{datetime.now().strftime('%d-%H:%M:%S')}] Processing
    ↪ {SRR}...", flush=True)

    # Obtención de los archivos fastq utilizando fasterq-dump, si no
    ↪ existen previamente
    if not (
        os.path.exists(fastq_single) or
        (os.path.exists(fastq_1) and os.path.exists(fastq_2))
    ):
        sb.run([
            "fasterq-dump", sra_file,
```

```
            "--split-files",
            # "--skip-technical",
            "--threads", "5",
            "--outdir", fastq_dir,
            "-t", "./tmp/"
        ], check=True)

        # Logs
        print(f"[{datetime.now().strftime('%d-%H:%M:%S')}] Finished
         ↪  processing {SRR}.", flush=True)
        print("-" * 50, flush=True)

print(f"[{datetime.now().strftime('%d-%H:%M:%S')}] All SRR files have
 ↪  been processed.", flush=True)
```

Evaluación de calidad

Una vez obtenidos los datos en formato `FASTQ`, se procedió a evaluar la calidad de las lecturas con `FASTQC`:

```
#| eval: false
#| echo: true

#!/usr/bin/env bash

set -euo pipefail # Si un comando falla, el script se detiene
shopt -s nullglob # Evita errores si no se encuentran archivos
 ↪  coincidentes

# Activando ambiente de conda para fastqc
source "$(conda info --base)/etc/profile.d/conda.sh"
conda activate multiqc

# Directorios de trabajo
DATA="./data"
FASTQ="$DATA/fastq"

mkdir -p "$FASTQ" ./logs/
```

```
# Evaluación de calidad de las lecturas utilizando fastqc en paralelo
nohup fastqc "$FASTQ"/*_1.fastq -o "${FASTQ}c_1" >
 ↪  ./logs/fastqc_1_explore.log 2>&1 &
nohup fastqc "$FASTQ"/*_2.fastq -o "${FASTQ}c_2" >
 ↪  ./logs/fastqc_2_explore.log 2>&1 &
```

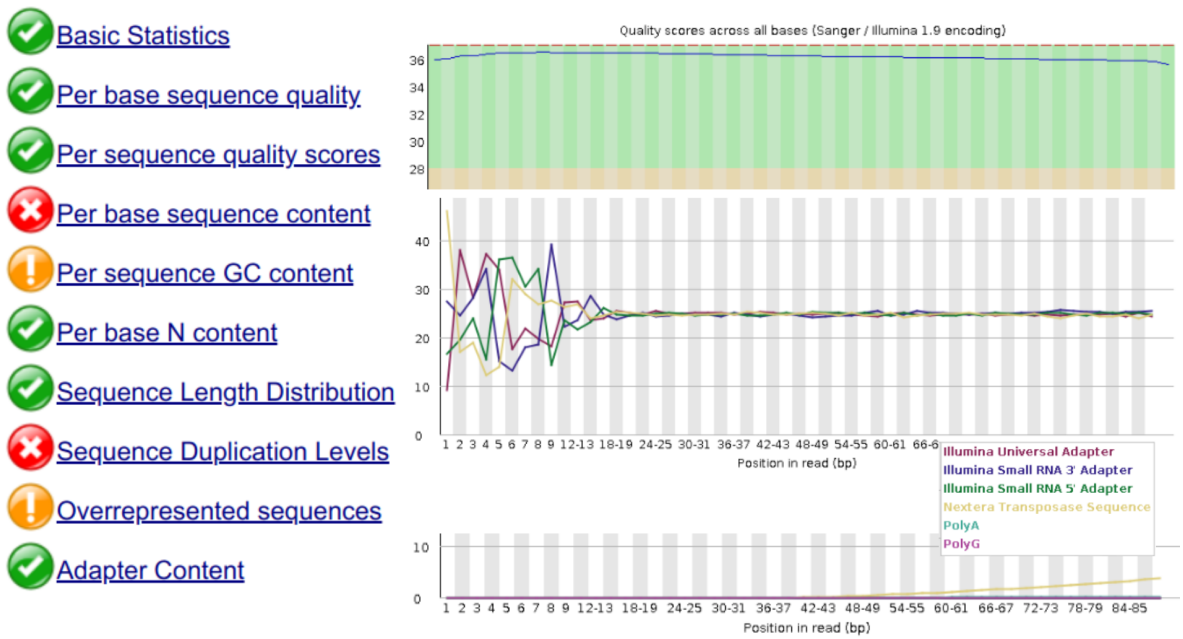

Figura 1: Resultados de FASTQC previo a la limpieza para la muestra SRR9126924_1.fastq

Como es posible observar en la **Figura 1**, las lecturas presentan una calidad de secuenciación buena, con un porcentaje de bases por encima de *Q30* en todos los casos. Ergo, se detectan adaptadores residuales de *Nextera Transpose Seq* y un sesgo de composición de bases al inicio de las lecturas (aproximadamente en las primeras 12-17 bp). Esta tendencia, ilustrada con una única muestra en la figura, se repite en las demás, lo que sugiere la necesidad de realizar una limpieza de las lecturas para mejorar la calidad de los datos antes del alineamiento.

Limpieza de lecturas

Haciendo uso de `fastp`, se procedió a realizar la limpieza de las lecturas, eliminando los adaptadores residuales y recortando los primeros 17 bp para eliminar el sesgo de composición

de bases al inicio de las lecturas. Acto seguido, se volvió a evaluar la calidad de las lecturas limpias utilizando `FastQC` para verificar la mejora en la calidad de las secuencias.

```
#| eval: false
#| echo: true

#!/usr/bin/env bash

# Cambio interno de ambientes de conda
source "$(conda info --base)/etc/profile.d/conda.sh"
conda activate fastp

# Configurando el match de patrones
set -euo pipefail
shopt -s nullglob

# Creación de carpetas
DATA="./data"
FASTQ="$DATA/fastq"
CLEAN="$DATA/fastp"
FASTQC_CLEAN_1="$DATA/fastqc_1_clean"
FASTQC_CLEAN_2="$DATA/fastqc_2_clean"
MULTIQC_DIR="$DATA/multiqc"

mkdir -p "$FASTQ" "$CLEAN" "$MULTIQC_DIR" "$FASTQC_CLEAN_1"
 ↪  "$FASTQC_CLEAN_2"

# Configuración de hilos y control de procesos
wks=3
threads_fastp=2
threads_fastqc=2

# Manejo de procesos en paralelo
pids=()
batch_processing() {
    if [[ ${#pids[@]} -eq $wks ]]; then
        wait "${pids[@]}"
        current_date_time="$(date "+%Y-%m-%d %H:%M:%S")"
        echo
 ↪  ">>>>
```

```
        echo "Lote de $wks procesos completados. Continuando con el
 ↪  script..."
        pids=()
    fi
}

echo "========== FASTP =========="

files=("$FASTQ"/SRR*_1.fastq)
erase_bp=17

for f in "${files[@]}"; do
    base=$(basename "$f" _1.fastq)

    # Limpieza PE por plataforma Ilumina NovaSeq
    fastp \
        -i "$FASTQ/${base}_1.fastq" \
        -I "$FASTQ/${base}_2.fastq" \
        -o "$CLEAN/${base}_clean_1.fastq" \
        -O "$CLEAN/${base}_clean_2.fastq" \
        -w "$threads_fastp" \
        --trim_poly_g \
        --trim_front1 $erase_bp \
        --trim_front2 $erase_bp \
        --detect_adapter_for_pe &

    # Guardado del JOBID para control de procesos
    pids+=("$!")
    batch_processing
done

# Manejo de procesos restantes
if [[ ${#pids[@]} -gt 0 ]]; then
    wait "${pids[@]}"
    pids=()
fi

echo "========== FASTQC SOBRE READS LIMPIOS =========="

conda activate multiqc
```

```
clean_files_1=("$CLEAN"/*_clean_1.fastq)

# Stats de la calidad resultante en las reads
for f1 in "${clean_files_1[@]}"; do
    base=$(basename "$f1" _clean_1.fastq)
    f2="$CLEAN/${base}_clean_2.fastq"

    fastqc \
        -o "$FASTQC_CLEAN_1" \
        -t "$threads_fastqc" \
        "$f1" &

    # Guardado del JOBID para control de procesos
    pids+=("$!")
    batch_processing

    # Si existe un archivo par, se procesa en paralelo, de lo contrario
 ↪  se muestra un mensaje de error
    if [[ -f "$f2" ]]; then
        fastqc \
            -o "$FASTQC_CLEAN_2" \
            -t "$threads_fastqc" \
            "$f2" &

        # Guardado del JOBID para control de procesos
        pids+=("$!")
        batch_processing
    else
        echo "Falta archivo par: $f2"
    fi
done

echo "========== MULTIQC ==========="

# Comparación global de las stats antes y después de la limpieza
multiqc "$DATA" -o "$MULTIQC_DIR"

echo "Proceso terminado."
```

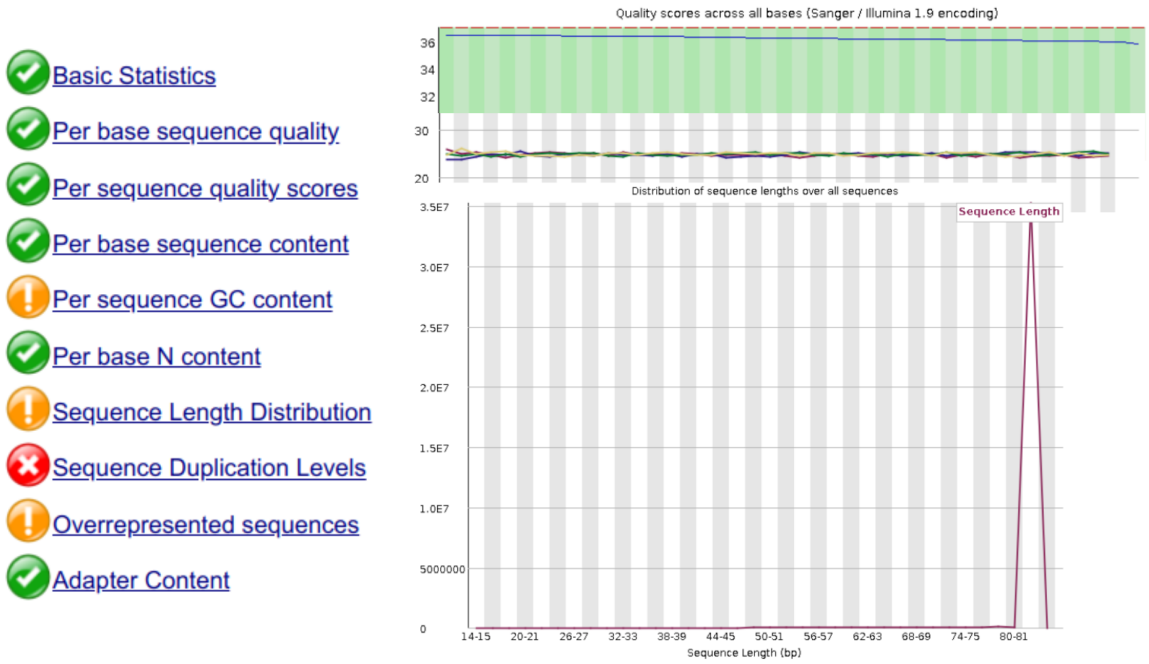

Figura 2: Resultados de FASTQC tras la limpieza para la muestra SRR9126924_1.fastq

Tras la limpieza, el sesgo de composición de bases al inicio de las lecturas se ha eliminado, y la calidad de las secuencias se mantiene con un porcentaje de bases por encima de *Q30* en prácticamente toda la longitud de las lecturas. Además, los adaptadores residuales han sido eliminados, lo que se refleja en la ausencia de picos correspondientes a los adaptadores en el gráfico de calidad. Asimismo, la distribución de la longitud de lecturas ha quedado en ~83 bp, acorde a lo esperado tras el recorte de los primeros 17 bp (ver **Figura 2**).

Alineamiento de lecturas

Se procedió a realizar el alineamiento de las lecturas limpias utilizando los alineadores STAR y HISAT2, con el fin de comparar su rendimiento y calidad de alineamiento en cada uno, tanto para *paired-end* (PE) como *single-end* (SE). Para ello, se utilizaron los índices correspondientes al genoma de referencia de ratón para cada alineador (ver carpeta `/export/space3/users/pablosm/2026-2/Transcriptomica_clase/data/indexes/` en el servidor *Chaac*), y se recuperó el porcentaje de lecturas alineadas y el tiempo de ejecución para cada muestra a partir de los `Log` de cada programa.

Para HISAT2, se empleó un esquema de alineamiento multihilo (`-p`) contra un genoma de referencia indexado (`-x`), utilizando archivos de entrada en formato `FASTQ` (`-U` para SE y

-1/-2 para PE) y generando archivos de salida en formato SAM (-S). Se habilitó la medición de tiempo de ejecución (-t), se excluyeron lecturas no alineadas (--no-unal) y se generaron reportes resumidos por muestra (--summary-file). Por su parte, STAR se ejecutó también en modo multihilo (--runThreadN), utilizando un índice de genoma previamente construido (--genomeDir) y archivos de entrada correspondientes a lecturas simples o pareadas (--readFilesIn). La salida se generó directamente en formato BAM ordenado por coordenadas (--outSAMtype BAM SortedByCoordinate), excluyendo lecturas no alineadas (--outSAMunmapped None). El tiempo de ejecución se registró por dos vías, mediante la herramienta externa /usr/bin/time y dentro de cada programa (para HISAT2 se utilizó el parámetro -t). Estas configuraciones permitieron comparar el desempeño de ambos alineadores en términos de eficiencia computacional y calidad de alineamiento.

```
#| eval: false
#| echo: true

#!/usr/bin/env bash

# Configuración del match de patrones
set -euo pipefail
shopt -s nullglob

# Creación de carpetas
DATA="./data"
LOGS="./logs"
RESULTS="./results"
CLEAN="${DATA}/fastp"
INDEXES="${DATA}/indexes"
STAR_PE="${RESULTS}/star_pe"
STAR_SE="${RESULTS}/star_se"
HISAT2_PE="${RESULTS}/hisat2_pe"
HISAT2_SE="${RESULTS}/hisat2_se"

mkdir -p "$STAR_PE" "$HISAT2_PE" "$HISAT2_SE" "$LOGS" "$STAR_SE"

# Índices de alineamiento
STAR_IDX="${INDEXES}/mm39.gencode.M36.star"
HISAT2_IDX="${INDEXES}/mm39.gencode.M36.hisat/mm39.gencode.M36.hisat"

[[ ! -f "${HISAT2_IDX}.1.ht2" ]] && echo "[ERROR] Índice HISAT2 no
 ↪  existente en $HISAT2_IDX" && exit 1
[[ ! -d "$STAR_IDX" ]] && echo "[ERROR] Directorio de índice STAR_PE no
 ↪  existente en $STAR_IDX" && exit 1
```

```
# Configuración de hilos
threads_star=8
threads_hisat2=8

echo "========== HISAT2 SE =========="

files=("$CLEAN"/SRR*_clean_1.fastq)

for f in "${files[@]}"; do
    base=$(basename "$f" _clean_1.fastq)

    # Alineamiento SE con HISAT2
    hisat2 \
        -p "$threads_hisat2" \
        -x "$HISAT2_IDX" \
        -U "$f" \
        -S "${HISAT2_SE}/${base}_hisat2_se.sam" \
        -t \
        --no-unal \
        --summary-file "${HISAT2_SE}/${base}_hisat2_se_summary.txt" \
        2> ${HISAT2_SE}/${base}_hisat2_se_time.txt

done

echo "========== HISAT2 PE =========="

for f in "${files[@]}"; do
    base=$(basename "$f" _clean_1.fastq)

    # Alineamiento PE con HISAT2
    hisat2 \
        -p "$threads_hisat2" \
        -x "$HISAT2_IDX" \
        -1 "$f" \
        -2 "$CLEAN/${base}_clean_2.fastq" \
        -S "$HISAT2_PE/${base}_hisat2_pe.sam" \
        -t \
        --no-unal \
        --summary-file "$HISAT2_PE/${base}_hisat2_pe_summary.txt" \
```

```
        2> ${HISAT2_PE}/${base}_hisat2_pe_time.txt

done

echo "========== STAR PE =========="

source "$(conda info --base)/etc/profile.d/conda.sh"
conda activate star

for f in "${files[@]}"; do
    base=$(basename "$f" _clean_1.fastq)

    # Alineamiento PE con STAR
    /usr/bin/time -f "%e" STAR \
        --runThreadN "$threads_star" \
        --genomeDir "$STAR_IDX" \
        --readFilesIn "$f" "${CLEAN}/${base}_clean_2.fastq" \
        --outFileNamePrefix "${STAR_PE}/${base}_star_pe_" \
        --outSAMtype BAM SortedByCoordinate \
        --outSAMunmapped None \
        2> ${STAR_PE}/${base}_star_pe_time.txt

done

echo "========== STAR SE =========="

for f in "${files[@]}"; do
    base=$(basename "$f" _clean_1.fastq)

    # Alineamiento SE con STAR
    /usr/bin/time -f "%e" STAR \
        --runThreadN "$threads_star" \
        --genomeDir "$STAR_IDX" \
        --readFilesIn "$f" \
        --outFileNamePrefix "${STAR_SE}/${base}_star_se_" \
        --outSAMtype BAM SortedByCoordinate \
        --outSAMunmapped None \
        2> ${STAR_SE}/${base}_star_se_time.txt

done
```

```
echo "Proceso terminado."
```

Las estadísticas de alineamiento obtenidas a partir de los archivos de resumen de HISAT2 y log de STAR se presentan en la **Tabla 1**. En ella se muestra el porcentaje de lecturas alineadas de forma única, alineadas a múltiples loci, la tasa de alineamiento global y el tiempo de ejecución para cada muestra y alineador.

Tabla 1: Estadísticas de alineamiento para HISAT2 y STAR.

	aligned_uniq	aligned_mult	overall_rate	time_sec
SRR9126754_hisat2_pe	73.15	5.47	89.36	368.0
SRR9126924_hisat2_pe	67.28	5.77	89.37	577.0
SRR9126950_hisat2_pe	63.78	5.68	86.64	361.0
SRR9127139_hisat2_pe	74.37	5.73	89.77	341.0
SRR9127244_hisat2_pe	73.01	4.94	90.22	154.0
SRR9127382_hisat2_pe	68.74	5.01	85.76	333.0
SRR9127568_hisat2_pe	58.31	5.92	83.53	315.0
SRR9126754_hisat2_se	78.24	11.55	89.79	174.0
SRR9126924_hisat2_se	76.77	12.79	89.56	236.0
SRR9126950_hisat2_se	73.79	13.23	87.02	164.0
SRR9127139_hisat2_se	78.54	11.48	90.02	161.0
SRR9127244_hisat2_se	81.09	11.16	92.25	69.0
SRR9127382_hisat2_se	74.52	11.65	86.16	176.0
SRR9127568_hisat2_se	69.76	13.87	83.63	149.0
SRR9126754_star_pe	79.34	6.07	85.41	261.0
SRR9126924_star_pe	73.22	6.57	79.79	347.0
SRR9126950_star_pe	70.35	6.62	76.97	240.0
SRR9127139_star_pe	80.00	6.32	86.32	223.0
SRR9127244_star_pe	81.56	6.12	87.68	121.0
SRR9127382_star_pe	75.59	5.68	81.27	215.0
SRR9127568_star_pe	65.49	6.98	72.47	217.0
SRR9126754_star_se	81.98	12.24	94.22	104.0
SRR9126924_star_se	80.45	13.86	94.31	151.0
SRR9126950_star_se	78.53	14.65	93.18	106.0
SRR9127139_star_se	82.29	12.38	94.67	98.0
SRR9127244_star_se	83.76	12.13	95.89	45.0
SRR9127382_star_se	79.97	13.13	93.10	103.0
SRR9127568_star_se	75.19	15.40	90.59	100.0

Tiempo de ejecución

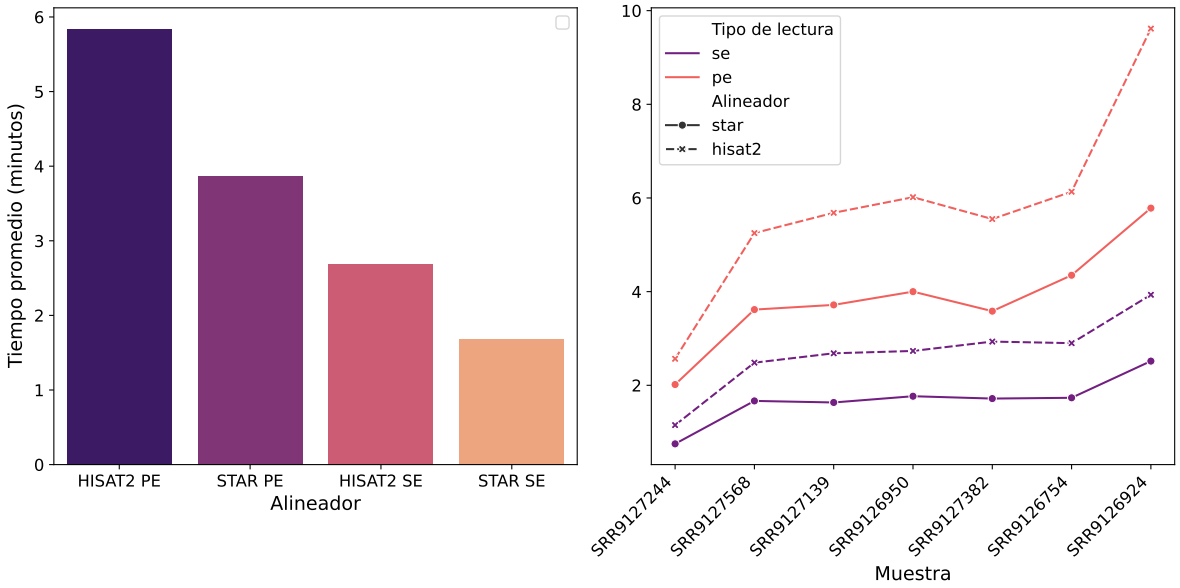

Figura 3: Tiempo de alineamiento.

Los alineamientos PE presentan un tiempo de ejecución mayor en comparación con los SE, lo cual es esperado debido a la mayor cantidad de datos a procesar. En cuanto a los alineadores, STAR muestra un tiempo de ejecución menor que HISAT2 en ambos casos (PE y SE), lo que sugiere una mayor eficiencia computacional (ver **Figura 3**, panel izquierdo). Esta diferencia podría atribuirse a las optimizaciones implementadas en STAR para el manejo de grandes volúmenes de datos y su capacidad para aprovechar múltiples hilos de procesamiento de manera más efectiva.

Entre muestras, el tiempo de alineamiento pareciera crecer a medida que crece el número de lecturas, aunque esta tendencia no es lineal, como se puede ver en el panel derecho de la **Figura 3**, en donde las muestras están ordenadas ascendentemente en función del número de lecturas. Esto podría deberse a variaciones en la complejidad del genoma de referencia, la calidad de las lecturas, la presencia de secuencias repetitivas que pueden afectar el tiempo de alineamiento o a la propia naturaleza de los algoritmos que, aunque con tendencias similares, resulta en un mejor desempeño para STAR en términos de tiempo de ejecución tanto para PE como SE.

Porcentaje de lecturas alineadas

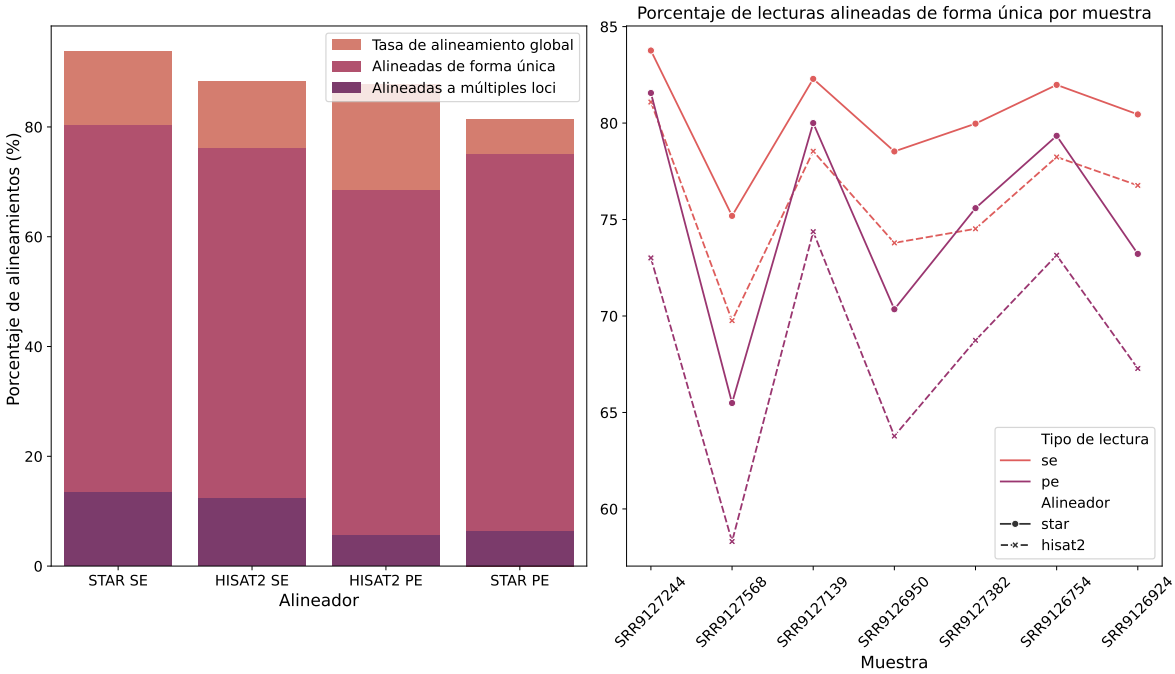

Figura 4: Porcentaje de lecturas alineadas.

Prácticamente todos los alineadores y tipos de lectura presentan una tasa de alineamiento global superior al 80%, lo cual es un buen resultado en términos de calidad de alineamiento (ver **Figura 4**, panel izquierdo). Mientras que la tasa de alineamientos únicos se encuentra entre el 75-80% para STAR y entre el 68-76% para HISAT2 (ver **Tabla 2**).

Tabla 2: Performance de los alineadores.

	Alineadas de forma única	Alineadas a múltiples loci	Tasa de alineamiento global
STAR SE	80.310000	13.398571	93.708571
HISAT2 SE	76.101429	12.247143	88.347143
HISAT2 PE	68.377143	5.502857	87.807143
STAR PE	75.078571	6.337143	81.415714

Asimismo, las lecturas SE presentan una tasa de alineamiento global ligeramente superior a las PE. Esto puede explicarse porque, en el caso de lecturas PE, el alineamiento está sujeto a criterios más estrictos relacionados con la concordancia entre ambos extremos de la lectura, lo que

puede reducir la proporción de alineamientos válidos en comparación con las lecturas SE. Aunado a ello, las lecturas PE parecen ser más conservadoras con la proporción de alineamientos a múltiples loci, como se logra ver en el panel izquierdo de la **Figura 4**, lo que sugiere que las lecturas PE podrían estar alineando de forma más específica a regiones del genoma, mientras que las SE podrían estar alineando a múltiples loci debido a la menor cantidad de información contextual disponible en cada lectura.

A nivel de muestras, no se observa una tendencia clara entre el número de lecturas y el porcentaje de alineamiento, lo que sugiere que otros factores como la calidad de las lecturas, la complejidad del genoma de referencia o la presencia de secuencias repetitivas podrían estar influyendo en la tasa de alineamiento más que el número total de lecturas (ver panel derecho de la **Figura 4**). Aquí, y como ya se había mencionado, STAR exhibe tasas de alineamientos únicos superiores a los alcanzados por HISAT2 en ambos tipos de lecturas.

Discusión

En suma, los resultados muestran diferencias claras entre los alineadores y los tipos de lectura tanto en términos de eficiencia computacional como de desempeño en el alineamiento. En primera instancia, los PE presentan mayores tiempos de ejecución en comparación con los SE, lo cual es consistente con el incremento en la cantidad de información a procesar y la necesidad de evaluar la concordancia entre ambos extremos de cada par de lecturas. En este contexto, STAR mostró consistentemente un menor tiempo de ejecución que HISAT2 para ambos tipos de lectura, lo que indica una mayor eficiencia computacional.

En términos de calidad de alineamiento, ambos alineadores presentan tasas globales superiores al 80%, lo cual indica un buen desempeño general. Sin embargo, al considerar la proporción de alineamientos únicos (que reflejan la especificidad del mapeo), STAR exhibe valores superiores en comparación con HISAT2 en ambos tipos de lecturas. Esto sugiere que STAR no solo es más rápido, sino que también logra una asignación más específica de las lecturas al genoma de referencia. Por otro lado, las lecturas SE muestran tasas de alineamiento global ligeramente mayores que las PE, lo cual puede explicarse por la menor restricción en los criterios de alineamiento, ya que las lecturas PE requieren cumplir condiciones adicionales de concordancia entre pares. En contraste, las lecturas PE tienden a presentar una menor proporción de alineamientos a múltiples loci, lo que sugiere una mayor especificidad derivada de la información adicional que proveen ambos extremos de la lectura.

Finalmente, a nivel de muestras, no se observa una relación directa entre el número de lecturas y la tasa de alineamiento, lo que indica que factores como la calidad de las lecturas o la complejidad del genoma tienen un mayor impacto en el desempeño del alineamiento que el tamaño del dataset en sí. En conjunto, estos resultados sugieren que **STAR presenta el mejor**

desempeño global al combinar mayor eficiencia computacional con una mayor proporción de alineamientos únicos, mientras que HISAT2, aunque más lento, mantiene un desempeño robusto y consistente en distintos escenarios.